INVESTIGACIÓN Y DESARROLLO

Algoritmos de Clasificación en Machine Learning Supervisado

Guía Completa con Ejemplos Prácticos y Código.

Roberto Molero.

1. ¿Qué es la Clasificación en Machine Learning?

Imagina que eres médico y recibes los resultados de un análisis de sangre: glucosa alta, presión elevada, historial familiar. Con tu experiencia, decides si ese paciente tiene o no diabetes. Eso es exactamente lo que hace un algoritmo de clasificación: aprende de ejemplos pasados para asignar una categoría o etiqueta a nuevos datos.

En términos más formales, la clasificación es una tarea de Machine Learning Supervisado donde el objetivo es aprender una función que mapea un conjunto de características de entrada (features) a una clase o categoría de salida discreta. El modelo aprende esa función durante el entrenamiento, usando datos ya etiquetados, y luego la aplica a datos nuevos que nunca ha visto.

¿Cuál es su propósito?

El propósito central es tomar decisiones automáticas y escalables. En lugar de que un humano tenga que revisar cada correo electrónico para saber si es spam, o cada imagen médica para detectar un tumor, un modelo entrenado puede hacerlo en milisegundos y a escala masiva, con una precisión a veces superior a la humana.

Ejemplo Práctico del Mundo Real: Detección de Fraude Bancario

Un banco procesa millones de transacciones por día. Cuando una tarjeta de crédito se usa a las 3 de la madrugada en otro país, en un comercio inusual, por un monto muy alto, podría ser fraude. Pero también podría ser un viajero legítimo.

Un modelo de clasificación analiza las características de cada transacción:

- Hora de la transacción
- País o ciudad donde se realiza
- Monto de la compra
- Comercio donde se hace
- Historial de compras del usuario
- Diferencia geográfica respecto a la última transacción

Con esos datos, el modelo aprende a clasificar cada transacción en dos categorías: FRAUDE o LEGÍTIMA. Si detecta fraude, bloquea la tarjeta automáticamente y alerta al cliente. Esto protege a millones de usuarios en tiempo real, algo imposible de hacer manualmente.

💡 Otros ejemplos cotidianos: filtros de spam en Gmail, diagnóstico médico por imágenes, moderación automática de contenido en redes sociales, aprobación de créditos bancarios y reconocimiento facial en teléfonos.

2. Clasificación Binaria vs. Multiclase

La principal diferencia está en el número de categorías posibles en las que puede caer cada observación.

Clasificación Binaria

Solo existen dos posibles respuestas. El modelo siempre tiene que decidir entre una de dos opciones. Piensa en un interruptor de luz: encendido o apagado, sí o no, verdadero o falso.

Ejemplos claros:

- ¿Este email es SPAM o NO SPAM?
- ¿Este paciente tiene CÁNCER o NO tiene cáncer?
- ¿Esta transacción es FRAUDULENTA o LEGÍTIMA?
- ¿Este cliente va a ABANDONAR el servicio (churn) o se va a QUEDAR?

💡 En clasificación binaria, una clase se llama 'positiva' (la que nos interesa detectar, ej: fraude) y otra 'negativa' (el caso normal, ej: transacción legítima). Esta distinción es clave para interpretar las métricas.

Clasificación Multiclase

Existen tres o más categorías posibles. El modelo debe elegir entre múltiples opciones. Como un clasificador postal que debe decidir a qué ciudad va una carta entre cientos de destinos posibles.

Ejemplos claros:

- Clasificar una imagen de animal: ¿es un PERRO, GATO, PÁJARO o PEZ?
- Identificar el idioma de un texto: ¿ESPAÑOL, INGLÉS, FRANCÉS, ALEMÁN?
- Reconocimiento de dígitos escritos a mano: 0, 1, 2, 3, 4, 5, 6, 7, 8 o 9
- Predicción de calidad de vino: MALO, REGULAR, BUENO, EXCELENTE

¿Cuándo usar cada uno?

Criterio	Binaria	Multiclase
Número de clases	Exactamente 2	3 o más
Complejidad del modelo	Más simple	Más compleja
Ejemplo de algoritmo	Regresión Logística estándar	Softmax, One-vs-Rest
Caso de uso típico	Detección de fraude, spam	Reconocimiento de imágenes
Métricas principales	AUC-ROC, Precision, Recall	Macro/Micro F1, Accuracy

3. Principales Algoritmos de Clasificación Supervisada

Cada algoritmo tiene su propia 'forma de pensar' el problema. No existe uno universalmente mejor: la elección depende del tipo de datos, el tamaño del dataset y lo que necesitas. A continuación se explica cada uno de manera clara, con analogías y código.

3.1 Regresión Logística

A pesar de su nombre, la Regresión Logística es un algoritmo de clasificación, no de regresión. El nombre es un legado histórico. Funciona calculando la probabilidad de que una observación pertenezca a una clase.

Analogía: Imagina que quieres predecir si un estudiante aprobará un examen basándote en las horas que estudió. Con pocas horas, la probabilidad de aprobar es baja. Con muchas horas, es alta. La función logística (o sigmoide) transforma cualquier número en un valor entre 0 y 1, que interpretamos como probabilidad.

¿Cuándo usarla? Es excelente como punto de partida (baseline), cuando los datos son linealmente separables, cuando necesitas interpretar qué variables son más importantes, y cuando el dataset no es enorme.

```python
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Cargar dataset de cáncer de mama (clasificación binaria)
data = load_breast_cancer()
X, y = data.data, data.target

# Dividir en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Crear y entrenar el modelo
model = LogisticRegression(max_iter=10000)
model.fit(X_train, y_train)

# Predecir y evaluar
predictions = model.predict(X_test)
print(f'Accuracy: {accuracy_score(y_test, predictions):.2f}')
# Output esperado: Accuracy: 0.96
```

3.2 Árboles de Decisión

Un Árbol de Decisión construye una serie de preguntas en forma de árbol. Cada nodo interior es una pregunta sobre una característica, cada rama es una respuesta posible, y cada hoja es una predicción final. Es como el juego de las 20 preguntas.

Analogía: Imagina que quieres decidir si llevar paraguas. Preguntas: ¿Hay nubes? Si no -> No lleves paraguas. Si sí -> ¿Dice la app que lloverá? Si sí -> Lleva paraguas. Ese proceso de decisiones encadenadas es exactamente un árbol de decisión.

¿Cuándo usarlo? Cuando necesitas que el modelo sea interpretable y explicable a personas no técnicas, cuando los datos tienen relaciones no lineales, y como base para modelos más poderosos como Random Forest.

```
from sklearn.tree import DecisionTreeClassifier, export_text

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split


# Dataset de flores Iris (3 clases: multiclase)

X, y = load_iris(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)


# Crear árbol con profundidad máxima de 3 (evita overfitting)

tree = DecisionTreeClassifier(max_depth=3, random_state=42)

tree.fit(X_train, y_train)


# Ver el árbol en texto

print(export_text(tree, feature_names=load_iris().feature_names))
```

```
print(f'Accuracy: {tree.score(X_test, y_test):.2f}')
```

3.3 Support Vector Machine (SVM)

SVM busca el hiperplano (una línea en 2D, un plano en 3D) que mejor separa las dos clases dejando el mayor 'margen' posible entre ellas. Los puntos más cercanos al hiperplano se llaman vectores de soporte, y son los que definen el límite.

Analogía: Imagina que tienes puntos rojos y azules en una hoja de papel y quieres dibujar una línea que los separe. SVM no dibuja cualquier línea, busca la que deja el mayor espacio libre entre los puntos de cada clase. Además, con el 'kernel trick' puede curvar ese límite para separar datos que no son linealmente separables.

¿Cuándo usarlo? En datasets de tamaño mediano con muchas dimensiones (texto, imágenes), cuando los datos tienen margen claro de separación, y cuando la precisión es más importante que la velocidad de entrenamiento.

```
from sklearn.svm import SVC

from sklearn.preprocessing import StandardScaler

from sklearn.pipeline import Pipeline

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split


X, y = load_breast_cancer(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)


# SVM necesita normalización de datos - usamos Pipeline
```

```
model = Pipeline([
    ('scaler', StandardScaler()),  # Normalizar features
    ('svm', SVC(kernel='rbf', C=1.0, gamma='scale', probability=True))
])
model.fit(X_train, y_train)


print(f'Accuracy SVM: {model.score(X_test, y_test):.2f}')
```

3.4 K-Nearest Neighbors (KNN)

KNN es quizás el algoritmo más intuitivo de todos: para clasificar un nuevo punto, simplemente busca los K puntos más cercanos en el conjunto de entrenamiento y le asigna la clase que tenga la mayoría. Literalmente, 'dime con quién andas y te diré quién eres'.

Analogía: Si llegas a vivir a un barrio nuevo y quieres saber si ese barrio es tranquilo o conflictivo, preguntas a tus 5 vecinos más cercanos. Si 4 de 5 dicen que es tranquilo, concluyes que lo es. KNN hace exactamente eso con datos.

Limitación importante: KNN es perezoso (lazy learning), no aprende una función explícita. Guarda todos los datos de entrenamiento y calcula distancias en cada predicción. Esto lo hace lento en datasets grandes.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score
import numpy as np


X, y = load_breast_cancer(return_X_y=True)
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)


# Probar diferentes valores de K para encontrar el óptimo

resultados = []

for k in range(1, 21):

    knn = KNeighborsClassifier(n_neighbors=k)

    scores = cross_val_score(knn, X_scaled, y, cv=5)

    resultados.append((k, scores.mean()))

    print(f'K={k:2d} -> Accuracy promedio: {scores.mean():.4f}')


best_k = max(resultados, key=lambda x: x[1])

print(f'\nMejor K: {best_k[0]} con Accuracy: {best_k[1]:.4f}')
```

3.5 Naive Bayes

Naive Bayes aplica el teorema de Bayes asumiendo que todas las características son independientes entre sí (de ahí 'naive' o ingenuo). Calcula la probabilidad de cada clase dado las características observadas y elige la más probable.

Analogía: Para clasificar un email como spam, Naive Bayes pregunta: ¿qué tan probable es que un email de spam contenga la palabra 'GRATIS'? ¿Y la palabra 'GANAR'? Multiplica esas probabilidades y compara con las de un email legítimo. Elige la clase con mayor probabilidad.

¿Cuándo usarlo? Es extraordinariamente rápido, funciona muy bien con texto (clasificación de documentos, análisis de sentimiento), y sorprendentemente efectivo incluso cuando la suposición de independencia no se cumple del todo.

```python
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# Ejemplo: clasificador de spam
emails = [
    'GRATIS gana dinero ya click aqui oferta',
    'Hola, te recuerdo nuestra reunion de mañana',
    'Premio exclusivo reclama tu regalo URGENTE',
    'Adjunto el informe que solicitaste ayer',
    'Gana millones desde casa sin esfuerzo gratis',
    'Confirmacion de tu pedido numero 4521',
]
labels = [1, 0, 1, 0, 1, 0]  # 1=spam, 0=legítimo

# Vectorizar el texto (convertir a números)
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(emails)

# Entrenar
nb = MultinomialNB()
nb.fit(X, labels)

# Predecir un nuevo email
nuevo = vectorizer.transform(['oferta especial gana premios gratis'])
```

```
print('Spam' if nb.predict(nuevo)[0] == 1 else 'Legítimo')
```

Algoritmo	Tipo de frontera	Velocidad	Interpretabilidad	Mejor para
Regresión Logística	Lineal	Muy rápida	Alta	Baseline, datos lineales
Árbol de Decisión	No lineal	Rápida	Muy alta	Reglas de negocio
SVM	Lineal/No lineal	Media	Baja	Alta dimensionalidad
KNN	No lineal	Lenta (test)	Media	Datasets pequeños
Naive Bayes	Probabilística	Muy rápida	Media	Texto, NLP

4. Métricas para Evaluar Modelos de Clasificación

Una métrica es como el termómetro de nuestro modelo: nos dice qué tan bien está funcionando, pero hay que elegir el termómetro correcto según la situación. Usar la métrica equivocada puede llevarte a conclusiones peligrosamente erróneas.

💡 Problema del dataset desbalanceado: Si el 99% de las transacciones son legítimas y 1% son fraude, un modelo que siempre diga 'legítimo' tendrá 99% de Accuracy... pero detectará 0% de los fraudes. Por eso necesitamos más métricas.

4.1 Accuracy (Exactitud)

Es la métrica más intuitiva: de todos los ejemplos, ¿qué porcentaje clasifiqué correctamente?

```
Accuracy = (TP + TN) / (TP + TN + FP + FN)


Donde:

  TP = True Positives  (predijo positivo, era positivo)

  TN = True Negatives  (predijo negativo, era negativo)

  FP = False Positives (predijo positivo, era negativo) <- Error Tipo I

  FN = False Negatives (predijo negativo, era positivo) <- Error Tipo II
```

¿Cuándo usarla? Cuando las clases están balanceadas (similar cantidad de ejemplos de cada clase). Por ejemplo, en el dataset MNIST de dígitos, cada número tiene aproximadamente el mismo número de muestras.

4.2 Precision (Precisión)

De todas las veces que el modelo dijo 'positivo', ¿cuántas realmente lo eran? Mide qué tan confiable es el modelo cuando hace una predicción positiva.

```
Precision = TP / (TP + FP)
```

```
Ejemplo: Si el modelo detectó 100 fraudes, pero solo 80 eran fraudes reales

  -> Precision = 80 / 100 = 0.80 (80%)
```

```
El 20% restante fueron falsas alarmas (clientes inocentes bloqueados)
```

¿Cuándo priorizarla? Cuando los falsos positivos son costosos. Ejemplo: en diagnóstico de cáncer, no quieres decirle a alguien sano que tiene cáncer y someterlo a quimioterapia innecesaria.

4.3 Recall (Sensibilidad / Exhaustividad)

De todos los casos positivos reales, ¿cuántos logró detectar el modelo? Mide qué tan bueno es el modelo para NO perder positivos.

```
Recall = TP / (TP + FN)
```

```
Ejemplo: Si había 100 fraudes reales y el modelo detectó solo 60

  -> Recall = 60 / 100 = 0.60 (60%)
```

```
El 40% restante fueron fraudes que pasaron desapercibidos
```

¿Cuándo priorizarlo? Cuando los falsos negativos son peligrosos. Ejemplo: en detección de enfermedades contagiosas, NO detectar un caso positivo puede costar vidas (el paciente no se aísla y contagia a otros).

4.4 F1-Score

Es la media armónica entre Precision y Recall. Es útil cuando quieres un balance entre ambas métricas y cuando el dataset está desbalanceado.

```
F1 = 2 * (Precision * Recall) / (Precision + Recall)


Ejemplo:

  Precision = 0.80, Recall = 0.60

  F1 = 2 * (0.80 * 0.60) / (0.80 + 0.60)

  F1 = 2 * 0.48 / 1.40

  F1 = 0.686 (68.6%)


# Por qué media armónica y no promedio simple?

  Promedio simple: (0.80 + 0.60) / 2 = 0.70 (70%)

  Media armónica: 0.686 (más conservadora, penaliza diferencias grandes)
```

Variante F-Beta: F2-Score da más peso al Recall (útil en medicina). F0.5-Score da más peso a la Precision (útil en spam donde prefieres no bloquear emails legítimos).

4.5 Matriz de Confusión

La Matriz de Confusión es una tabla que muestra visualmente cómo se distribuyen los errores del modelo. Para clasificación binaria es una tabla 2x2; para multiclase es NxN donde N es el número de clases.

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

import matplotlib.pyplot as plt

from sklearn.linear_model import LogisticRegression

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split


X, y = load_breast_cancer(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)


model = LogisticRegression(max_iter=10000)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)


cm = confusion_matrix(y_test, y_pred)

print('Matriz de Confusión:')

print('                Predicho: No  |  Predicho: Sí')

print(f'Real: No       {cm[0][0]:>12}   |   {cm[0][1]:<10}')

print(f'Real: Sí       {cm[1][0]:>12}   |   {cm[1][1]:<10}')


# Visualización gráfica

disp = ConfusionMatrixDisplay(confusion_matrix=cm)

disp.plot()

plt.title('Matriz de Confusión')

plt.savefig('confusion_matrix.png')
```

💡 **Lectura de la matriz: La diagonal principal (de arriba-izquierda a abajo-derecha) muestra los aciertos. Todo lo que está fuera de la diagonal son errores. En un modelo perfecto, solo la diagonal tendría valores.**

4.6 ROC-AUC

ROC (Receiver Operating Characteristic) es una curva que muestra el rendimiento del modelo en todos los umbrales de decisión posibles. AUC (Area Under the Curve) es el área bajo esa curva, que resume en un solo número qué tan bueno es el modelo.

Interpretación del AUC: AUC = 1.0 es perfecto. AUC = 0.5 es lo mismo que adivinar al azar. AUC > 0.9 se considera excelente. AUC entre 0.7-0.9 es bueno. AUC < 0.7 necesita mejoras.

```
from sklearn.metrics import roc_auc_score, roc_curve

from sklearn.linear_model import LogisticRegression

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler


X, y = load_breast_cancer(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)


scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)


model = LogisticRegression(max_iter=10000)

model.fit(X_train, y_train)
```

```
# predict_proba da probabilidades, no solo la clase

y_proba = model.predict_proba(X_test)[:, 1]  # probabilidad de clase positiva



auc = roc_auc_score(y_test, y_proba)

print(f'AUC-ROC: {auc:.4f}')  # Esperamos ~0.99 para este dataset
```

5. Feature Engineering y Feature Selection

Los datos crudos rara vez vienen en la forma perfecta para un modelo de ML. El feature engineering y feature selection son el arte de transformar y seleccionar las variables correctas para que el modelo pueda aprender mejor.

Una frase célebre en ML: 'Garbage in, garbage out' (si entras datos basura, obtienes resultados basura). Estas técnicas son fundamentales para evitar ese problema.

Feature Engineering: Crear Variables Mejores

Es el proceso de transformar datos crudos en características que el modelo pueda entender y aprovechar mejor. Implica creatividad, conocimiento del dominio y experimentación.

Tipos principales de transformaciones:

- Normalización y escalado: Llevar todas las variables a la misma escala (0-1 o media 0, desviación 1). Fundamental para KNN, SVM y regresión logística.
- Codificación de variables categóricas: Los modelos trabajan con números. Hay que convertir categorías como 'rojo', 'azul', 'verde' a representaciones numéricas.
- Creación de variables derivadas: Combinar variables existentes para crear nuevas con más poder predictivo.
- Tratamiento de fechas: Extraer día de la semana, mes, hora, es fin de semana, etc.
- Manejo de valores faltantes: Imputar con la media, mediana, o usar modelos especiales.
-

```
import pandas as pd
from sklearn.preprocessing import StandardScaler, LabelEncoder

# Dataset ejemplo: información de clientes
df = pd.DataFrame({
    'edad': [25, 34, 45, 22, None],
    'salario': [30000, 55000, 80000, 25000, 60000],
    'ciudad': ['Madrid', 'Barcelona', 'Madrid', 'Valencia', 'Barcelona'],
    'fecha_registro': ['2020-01-15', '2019-06-20', '2021-03-01', '2022-11-10',
'2020-08-05']
})

# 1. Imputar valores faltantes con la mediana
df['edad'].fillna(df['edad'].median(), inplace=True)

# 2. Crear variable nueva: ratio salario/edad
df['salario_por_edad'] = df['salario'] / df['edad']

# 3. Codificar variable categórica (One-Hot Encoding)
df = pd.get_dummies(df, columns=['ciudad'], prefix='ciudad')

# 4. Extraer features de fecha
df['fecha_registro'] = pd.to_datetime(df['fecha_registro'])
df['anio_registro'] = df['fecha_registro'].dt.year
df['mes_registro'] = df['fecha_registro'].dt.month
df['antiguedad_dias'] = (pd.Timestamp.now() - df['fecha_registro']).dt.days

# 5. Escalar variables numéricas
scaler = StandardScaler()
cols_numericas = ['edad', 'salario', 'salario_por_edad']
df[cols_numericas] = scaler.fit_transform(df[cols_numericas])

print(df.head())
```

Feature Selection: Quedarse con lo Importante

No todas las variables ayudan al modelo. Algunas son irrelevantes, otras están duplicadas, y demasiadas variables pueden causar 'curse of dimensionality' (maldición de la dimensionalidad). Feature Selection elimina el ruido.

Principales métodos:

- Filtrado estadístico (Filter): Mide la correlación entre cada variable y la clase objetivo. Métodos como chi-cuadrado, información mutua o correlación de Pearson.
- Métodos Wrapper: Entrenan el modelo con subconjuntos de variables y eligen el mejor. Más preciso pero computacionalmente costoso.

- Métodos Embedded: El propio modelo selecciona variables durante el entrenamiento. Ej: Random Forest tiene importancia de variables incorporada.

```
from sklearn.feature_selection import SelectKBest, f_classif, RFE
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_breast_cancer
import pandas as pd
import numpy as np

data = load_breast_cancer()
X, y = data.data, data.target
feature_names = data.feature_names

# ---- Método 1: Filtrado estadístico (ANOVA F-test) ----
selector = SelectKBest(f_classif, k=10)  # Seleccionar top 10
X_selected = selector.fit_transform(X, y)
mask = selector.get_support()
print('Top 10 features (ANOVA):')
print([f for f, m in zip(feature_names, mask) if m])

# ---- Método 2: Importancia de variables con Random Forest ----
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X, y)

importancias = pd.Series(rf.feature_importances_, index=feature_names)
print('\nTop 5 variables más importantes:')
print(importancias.sort_values(ascending=False).head())
```

💡 Regla práctica: Empieza con todas las variables, entrena un Random Forest, revisa la importancia de cada variable y elimina las que tienen importancia cercana a cero. Esto suele mejorar el modelo y acelerarlo significativamente.

7. Hiperparámetros y su Optimización

Existe una distinción fundamental que todo profesional de ML debe conocer: la diferencia entre parámetros e hiperparámetros.

Parámetros: Son los valores que el modelo aprende durante el entrenamiento de forma automática. Por ejemplo, los pesos (w) de una red neuronal o los coeficientes de una regresión logística.

Hiperparámetros: Son las configuraciones que TÚ defines ANTES de entrenar el modelo. El modelo no los aprende: tú los ajustas. Ejemplos: la profundidad máxima de un árbol, el número de vecinos K en KNN, o la tasa de aprendizaje en una red neuronal.

Algoritmo	Hiperparámetro	¿Qué controla?	Valores típicos
K-Nearest Neighbors	n_neighbors (K)	Cantidad de vecinos a consultar	1, 3, 5, 7, 10, 15
Árbol de Decisión	max_depth	Profundidad máxima del árbol	3, 5, 10, None
SVM	C	Tolerancia al error (regularización)	0.1, 1, 10, 100
SVM	kernel	Tipo de frontera de decisión	linear, rbf, poly
Random Forest	n_estimators	Número de árboles en el bosque	50, 100, 200, 500
XGBoost	learning_rate	Velocidad de aprendizaje	0.01, 0.05, 0.1, 0.3

Grid Search: Búsqueda Exhaustiva

Grid Search prueba TODAS las combinaciones posibles de hiperparámetros que le especifiques. Es como probar todos los sabores de pizza del menú: garantizas encontrar tu favorito, pero puede llevar mucho tiempo si el menú es extenso.

```
from sklearn.model_selection import GridSearchCV

from sklearn.svm import SVC

from sklearn.datasets import load_breast_cancer

from sklearn.preprocessing import StandardScaler

from sklearn.pipeline import Pipeline

import time
```

```
X, y = load_breast_cancer(return_X_y=True)


# Definir el pipeline
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC(random_state=42))
])


# Definir la 'rejilla' de hiperparámetros a probar
# Esto probará 4 x 2 x 3 = 24 combinaciones
param_grid = {
    'svm__C':      [0.1, 1, 10, 100],
    'svm__kernel': ['rbf', 'linear'],
    'svm__gamma':  ['scale', 'auto', 0.001]
}


# cv=5 significa validación cruzada con 5 particiones
# Por lo tanto: 24 combinaciones x 5 folds = 120 entrenamientos
grid_search = GridSearchCV(
    pipeline,
    param_grid,
    cv=5,
    scoring='f1',  # Optimizamos F1-Score
    n_jobs=-1,     # Usar todos los núcleos del procesador
    verbose=1
)
```

```
start = time.time()

grid_search.fit(X, y)

end = time.time()



print(f'Tiempo: {end-start:.1f} segundos')

print(f'Mejores hiperparámetros: {grid_search.best_params_}')

print(f'Mejor F1-Score: {grid_search.best_score_:.4f}')
```

Desventaja de Grid Search: Si tienes muchos hiperparámetros o rangos amplios, el número de combinaciones explota exponencialmente. 5 hiperparámetros con 5 valores cada uno = 3,125 combinaciones. Con 10 hiperparámetros = millones.

Random Search: Búsqueda Aleatoria

En lugar de probar todas las combinaciones, Random Search muestrea aleatoriamente combinaciones del espacio de hiperparámetros. La investigación de Bergstra & Bengio (2012) demostró que Random Search suele encontrar resultados igual de buenos o mejores que Grid Search en una fracción del tiempo.

Analogía: Si Grid Search es probar cada asiento del estadio para encontrar el de mejor vista, Random Search es sentarte en 50 asientos elegidos al azar y quedarte con el mejor. Estadísticamente, tienes alta probabilidad de encontrar un asiento excelente sin revisar los 50,000.

```
from sklearn.model_selection import RandomizedSearchCV

from sklearn.ensemble import RandomForestClassifier

from scipy.stats import randint, uniform

from sklearn.datasets import load_breast_cancer
```

```
X, y = load_breast_cancer(return_X_y=True)


# Definir distribuciones de probabilidad para cada hiperparámetro
# Esto permite explorar espacios continuos, no solo valores discretos
param_distributions = {
    'n_estimators':       randint(50, 500),      # Entre 50 y 500 árboles
    'max_depth':          [None, 5, 10, 15, 20], # Profundidad o None (sin límite)
    'min_samples_split':  randint(2, 20),         # Mínimo de muestras para dividir
    'min_samples_leaf':   randint(1, 10),          # Mínimo de muestras en hoja
    'max_features':       ['sqrt', 'log2', None], # Variables por split
    'bootstrap':          [True, False]           # Usar bootstrap o no
}


# n_iter=50: Solo probar 50 combinaciones aleatorias
# En lugar de potencialmente miles con GridSearch
random_search = RandomizedSearchCV(
    RandomForestClassifier(random_state=42),
    param_distributions,
    n_iter=50,        # Probar 50 combinaciones aleatorias
    cv=5,
    scoring='roc_auc',
    n_jobs=-1,
    random_state=42,
    verbose=1
)
random_search.fit(X, y)
```

```
print(f'Mejores hiperparámetros encontrados:')

for param, value in random_search.best_params_.items():

    print(f'  {param}: {value}')

print(f'Mejor AUC-ROC: {random_search.best_score_:.4f}')
```

Comparación: Grid Search vs Random Search

Aspecto	Grid Search	Random Search
Cobertura	Exhaustiva (toda la rejilla)	Aleatoria (muestra representativa)
Velocidad	Lento con muchos params	Más rápido
Garantía de óptimo	Sí (dentro de la rejilla)	No, pero suele encontrar buenas soluciones
Espacios continuos	No (solo valores discretos)	Sí (distribuciones de probabilidad)
Uso recomendado	Pocos hiperparámetros (<4)	Muchos hiperparámetros o inicial
Recursos	Alto consumo computacional	Controlable con n_iter

💡 **Consejo profesional: El flujo ideal es: (1) Usa Random Search para explorar un espacio amplio y encontrar regiones prometedoras. (2) Luego usa Grid Search en una rejilla fina alrededor de los mejores valores encontrados. Este enfoque de dos fases da los mejores resultados con el menor costo computacional.**

Técnicas Avanzadas: Más Allá de Grid y Random

Para proyectos más exigentes, existen métodos más sofisticados:

- Bayesian Optimization (Optuna, Hyperopt): Usa el historial de evaluaciones anteriores para decidir inteligentemente qué combinación probar a continuación. Es como un investigador que aprende de sus experimentos previos.
- Halving Search (HalvingGridSearchCV): Comienza con muchos candidatos y pocos datos, elimina los peores y da más datos a los mejores en cada ronda. Mucho más eficiente.

```python
# Ejemplo con Optuna (Bayesian Optimization)
# pip install optuna
import optuna
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score
from sklearn.datasets import load_breast_cancer
optuna.logging.set_verbosity(optuna.logging.WARNING)

X, y = load_breast_cancer(return_X_y=True)

def objective(trial):
    # Optuna sugiere hiperparámetros de forma inteligente
    n_estimators = trial.suggest_int('n_estimators', 50, 300)
    max_depth = trial.suggest_int('max_depth', 3, 15)
    min_samples_split = trial.suggest_int('min_samples_split', 2, 10)

    model = RandomForestClassifier(
        n_estimators=n_estimators,
        max_depth=max_depth,
        min_samples_split=min_samples_split,
        random_state=42
    )
    score = cross_val_score(model, X, y, cv=5, scoring='roc_auc').mean()
    return score  # Optuna maximizará este valor

# Crear estudio y optimizar
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50)

print(f'Mejores hiperparámetros: {study.best_params}')
print(f'Mejor AUC-ROC: {study.best_value:.4f}')
```